Driftpunkt installationsguider

Version 1.0.10 · samlad PDF för installation, Debian-server och NAS/Docker

Skapad från public repo 2026-04-28 08:14 UTC.

Innehåll

- General installation and deployment — `installation_and_deployment.md`
- Debian server setup — `debian_server_setup.md`
- NAS Docker setup — `nas_docker_setup.md`

General installation and deployment

Källa: `docs/installation_and_deployment.md`

Installation och driftsättning

Lokala förutsättningar

- PHP 8.4 eller senare
- Composer 2
- MariaDB 10.11 eller senare
- PHP-tilläggen `ctype`, `curl`, `iconv`, `intl`, `mbstring`, `pdo`, `pdo_mysql`, `xml` och `zip`
- `DATABASE_URL` mot MariaDB utanför `APP_ENV=test`; vid `mysql://` ska `serverVersion` innehålla `mariadb`
- skrivbar `var/`-katalog för cache, loggar, uppladdningar och delade driftfiler
- webbserverns dokumentrot ska peka på `htdocs/`
- valfritt: Symfony CLI

Webbrot och `/htdocs`

Driftpunkt använder `htdocs/` som aktiv dokumentrot. En server eller NAS som redan exponerar en mapp som heter `/htdocs` ska därför kunna köra sajten utan att lägga `/public` i URL eller webbserverkonfiguration.

Rekommenderad struktur:

```
/var/www/driftpunkt/
├── bin/
├── config/
├── htdocs/
│   ├── .htaccess
│   ├── index.php
│   └── assets/
├── src/
├── templates/
├── var/
└── vendor/
```

Apache/NAS bör peka `DocumentRoot` mot applikationens `htdocs/` när du kan välja webbrot själv. `public/` kan finnas kvar som bakåtkompatibel rot under en övergång, men nya installationer och releasepaket ska verifieras mot `htdocs/`.

På webbhotell där den publika katalogen redan heter `/htdocs` och inte kan ändras kan hela installationspaketet packas upp direkt i den yttre `/htdocs`-mappen:

```
/htdocs/
├── .htaccess
├── index.php
├── bin/
├── config/
├── htdocs/
│   └── assets/
├── src/
├── templates/
├── var/
└── vendor/
```

I det läget använder root-`.htaccess` paketets root-`index.php`, blockerar direkta anrop till känsliga mappar som `config/`, `src/`, `vendor/` och `var/`, och serverar `/assets/...` från den interna `htdocs/assets/`. Det kräver att Apache har `mod_rewrite` och tillåter `.htaccess`.

Lokal installation

1. Installera beroenden:

```
composer install
```

2. Kontrollera miljö.

Standardvärdena i `.env` fungerar mot MariaDB i rootens `compose.yaml`. Skapa `.env.local` om du behöver ändra `APP_SECRET`, `DEFAULT_URI`, `DATABASE_URL`, `MAILER_DSN` eller andra lokala värden.

Exempel på lokal MariaDB-URL:

```
DATABASE_URL="mysql://driftpunkt:driftpunkt@127.0.0.1:33060/driftpunkt?serverVersion=mariadb-11.8.6&charset=utf8mb4"
```

3. Initiera databasen.

För en helt ny MariaDB-installation:

```
php bin/console app:install:fresh
```

Kommandot skapar schema direkt från entiteterna, initierar migrationsmetadata och markerar befintliga migrationer som baseline för den nya installationen. Standardkonton för superadmin och admin säkerställs alltid. Standardkonton för tekniker och kund skapas också automatiskt, om du inte anger `--skip-test-accounts`. Standardkonton markeras för lösenordsbyte vid första inloggningen.

Om du i stället arbetar mot en befintlig databas där migrationshistorik redan ska följas:

```
php bin/console doctrine:migrations:migrate -n
```

4. Skapa konto vid behov:

```
php bin/console app:create-test-accounts
```

eller:

```
php bin/console app:create-admin dinmail@example.com DittLösenord123 Förnamn Efternamn super_admin
```

5. Starta webbserver:

```
symfony server:start
```

Om Symfony CLI saknas:

```
php -S 127.0.0.1:8000 -t htdocs
```

Docker och lokala hjälptjänster

Repo innehåller `compose.yaml` och `compose.override.yaml` med MariaDB och mailtestning för lokal Docker-baserad utveckling.

Vanliga Docker-kommandon från projektets rotmapp:

```
docker compose --env-file .env -f compose.yaml up -d
docker compose --env-file .env -f compose.yaml ps
docker compose --env-file .env -f compose.yaml logs --tail=150 app
```

Om tjänstenamnen avviker mellan miljöer, kontrollera dem med:

```
docker compose --env-file .env -f compose.yaml config --services
```

Produktionsdrift

En fungerande driftsättning bör innehålla:

- webbserver som pekar på `htdocs/`
- alternativt fast webbhotellrot `/htdocs` där paketets root-`.htaccess` är aktiv
- skrivbar `var/`
- skrivbar `htdocs/assets/branding` om logotyp och branding ska kunna ändras från adminytan
- MariaDB-databas med korrekt `serverVersion` i `DATABASE_URL`
- standard-superadmin och minst ett vanligt admin-konto
- process för schemalagda jobb
- worker för köade koduppdateringar om adminytans uppdateringsflöde används
- loggrotation eller motsvarande retention för `var/log/*.log`

Schemalagda jobb bör minst omfatta mailpolling, SLA-kontroll, bilagearkivering och månadsrapporter om rapportmail används. Månadsrapporten kan exempelvis köras efter månadsskifte:

```
php bin/console app:reports:send-monthly --env=prod
```

Om adminytans uppdateringsflöde ska användas för att applicera kodpaket måste PHP/webbserver-användaren, normalt `www-data`, också kunna skriva till koddelarna som byts vid uppdatering: `bin/`, `config/`, `htdocs/`, `migrations/`, `public/`, `src/`, `templates/`, `composer.json`, `composer.lock` och `symfony.lock`.

På en mer låst produktion kan kodfilerna i stället ägas av `root` och bara `var/` vara skrivbar för `www-data`, men då ska koduppdateringar göras via SSH/deploy-script i stället för via adminytan.

För en konkret Debian-installation, använd:

- `docs/debian_server_setup.md`
- `deploy/debian/app.env.example`
- `deploy/debian/apache-driftpunkt.conf`
- `deploy/debian/setup.sh`

Automatisk hantering av köade uppdateringar

Driftpunkt kan lägga koduppdateringar i kö från adminytan. En köad uppdatering sparas som JSON i `var/code_update_runs`. För att uppdateringen inte ska bli liggande som queued behöver produktionen ha en schemalagd worker.

Rekommenderad modell:

```
Adminyta köar uppdatering
        ↓
cron/systemd timer kör worker
        ↓
worker hittar queued update
        ↓
worker kör app:code-update:apply-run
        ↓
status blir completed eller failed
```

Webbprocessen ska helst inte vara enda mekanismen för att starta själva uppdateringen. Det är säkrare att låta cron, systemd timer eller Docker-scheduler plocka upp jobbet.

Docker/NAS: cron-worker

Skapa scriptet i projektets rotmapp:

```
nano /share/Docker/driftpunkt/run-pending-updates.sh
```

Innehåll:

```
#!/bin/sh

cd /share/Docker/driftpunkt || exit 1

RUN_ID=$(docker compose --env-file .env -f compose.yaml exec -T app php -r '
$dir = "var/code_update_runs";
if (!is_dir($dir)) { exit; }

$files = glob($dir . "/*.json");
rsort($files);

foreach ($files as $file) {
    $data = json_decode(file_get_contents($file), true);
    if (($data["status"] ?? "") === "queued") {
        echo basename($file, ".json");
        exit;
    }
}
')

if [ -n "$RUN_ID" ]; then
    echo "Kör Driftpunkt-uppdatering: $RUN_ID"
    docker compose --env-file .env -f compose.yaml exec -T --user www-data app php bin/console
app:code-update:apply-run "$RUN_ID" --env=prod
else
    echo "Ingen köad Driftpunkt-uppdatering hittades."
fi
```

Gör scriptet körbart:

```
chmod +x /share/Docker/driftpunkt/run-pending-updates.sh
```

Testa scriptet:

```
/share/Docker/driftpunkt/run-pending-updates.sh
```

Om ingen uppdatering väntar ska svaret vara:

```
Ingen köad Driftpunkt-uppdatering hittades.
```

Lägg in scriptet i root-cron:

```
sudo crontab -e
```

Lägg till:

```
* * * * * /share/Docker/driftpunkt/run-pending-updates.sh >> /share/Docker/driftpunkt/var/
background_jobs/auto_update_cron.log 2>&1
```

Kontrollera:

```
sudo crontab -l
tail -n 50 /share/Docker/driftpunkt/var/background_jobs/auto_update_cron.log
```

Om NAS:en inte stödjer vanlig `crontab -e`, skapa motsvarande schemalagd uppgift i NAS:ens webbgränssnitt och kör samma script var 1–5 minut.

Debian utan Docker: systemd timer

På en vanlig Debian-server utan Docker rekommenderas `systemd timer`. Se `docs/debian_server_setup.md` för komplett exempel. Grundprincipen är:

```
sudo systemctl enable --now driftpunkt-update-worker.timer
systemctl status driftpunkt-update-worker.timer
journalctl -u driftpunkt-update-worker.service -n 100 --no-pager
```

Rekommenderad efter-installation

Verifiera följande efter installation:

- [ ] webbgränssnittet går att nå
- [ ] webbservern pekar på `htdocs/` och `/` visar Driftpunkts startsida

- [ ] om hela paketet ligger direkt i yttre `/htdocs`: kontrollera att `/config/`, `/src/`, `/vendor/` och `/var/` ger 403 eller inte är åtkomliga
- [ ] inloggning fungerar
- [ ] databasanslutning fungerar
- [ ] standard-superadmin och vanligt admin-konto finns
- [ ] `var/` är skrivbar för rätt användare
- [ ] mailservrar och supportinkorgar är konfigurerade vid behov
- [ ] polling, SLA-jobb och bilagearkivering är aktiverade
- [ ] automatisk worker för köade uppdateringar är aktiverad om adminytans uppdateringsflöde används
- [ ] backup och restore har testats
- [ ] driftstatussidan är verifierad

Uppgradering

Bygg eller leverera uppgraderingspaket:

```
php bin/console app:release:build-packages --type=upgrade
```

Uppgraderingspaket innehåller `vendor/` för att NAS- och Debian-installationer inte ska behöva hämta PHP-paket från internet mitt i en uppdatering.

Releasepaketen tar bara med installations- och uppdateringsnära dokumentation från `docs/`: huvudguiden, Debian-guiden, NAS-guiden, mail-/jobbdrift, bilagearkivering, säkerhetskrav, kända driftbegränsningar och addon-releaseguiden. Produktplaner, interna specar, skärmbilder och designunderlag ska inte följa med i installations- eller uppgraderingszipparna.

Rekommenderad ordning i drift:

1. skapa backup
2. aktivera underhållsläge
3. stagea/applicera uppdateringspaket
4. låt uppdateringsflödet köra Composer, MariaDB-migrationer och cache-rensning
5. verifiera att update-worker markerar körningen som `completed`
6. verifiera inloggning, tickets och mail
7. avaktivera underhållsläge

Manuell körning av en specifik update-run:

```
php bin/console app:code-update:apply-run RUN_ID --env=prod
```

Docker/NAS:

```
docker compose --env-file .env -f compose.yaml exec -T --user www-data app php bin/console app:
code-update:apply-run RUN_ID --env=prod
```

Kontrollera status för en körning:

```
cat var/code_update_runs/RUN_ID.json
```

Debian server setup

Källa: `docs/debian_server_setup.md`

Debian-server: krav och installation

Den här guiden är den kortaste vägen till en vanlig Debian-installation utan Docker. Den utgår från att appen ligger i `/var/www/driftpunkt`, körs av `www-data` och exponeras via Apache.

Krav

Minimikrav:

- Debian-server med shell-access och `sudo`
- PHP 8.4 eller senare
- Composer 2
- Apache 2 med `mod_rewrite`
- MariaDB 10.11 eller senare
- Skrivbar `var/`-mapp för cache, loggar, bilagor och delade filer
- Apache DocumentRoot mot `/var/www/driftpunkt/htdocs`
- systemd timers eller cron för bakgrundsjobb
- systemd timer för köade koduppdateringar om adminytans uppdateringsflöde används

Krav för webbaserade uppdateringar från adminytan:

- PHP/webbserver-användaren, normalt `www-data`, måste kunna skriva till applikationskoden som uppdateringspaketet hanterar: `bin/`, `config/`, `htdocs/`, `migrations/`, `public/`, `src/`, `templates/`, `composer.json`, `composer.lock` och `symfony.lock`
- Om servern ska vara hårdare låst med kodfiler ägda av `root` ska adminytans uppdateringsflöde inte användas för kodbyte. Uppdatera då i stället via SSH/deploy-script och låt bara `var/` vara skrivbar för `www-data`.

För en installation där adminytan ska kunna applicera uppdateringspaket:

```
sudo chown -R www-data:www-data /var/www/driftpunkt
sudo chmod -R u+rwX /var/www/driftpunkt
```

För en mer låst server:

```
sudo chown -R root:www-data /var/www/driftpunkt
sudo chown -R www-data:www-data /var/www/driftpunkt/var /var/www/driftpunkt/htdocs/assets
sudo find /var/www/driftpunkt -type d -exec chmod 755 {} \;
sudo find /var/www/driftpunkt -type f -exec chmod 644 {} \;
```

I den låsta modellen ska koduppdateringar göras via SSH/deploy-script, inte via adminytans kodapplicering.

Rekommenderade PHP-tillägg:

- `ctype`
- `curl`
- `iconv`
- `intl`

- `mbstring`
- `pdo`
- `pdo_mysql`
- `xml`
- `zip`
- `opcache`

Databasval:

- MariaDB är standard för drift, Docker/NAS och Debian.
- SQLite används bara när `APP_ENV=test`; live-, Docker- och Debian-miljöer ska köra MariaDB.

Snabb installation

1. Kopiera koden till servern:

```
sudo mkdir -p /var/www/driftpunkt
sudo chown "$USER":www-data /var/www/driftpunkt
git clone <repo-url> /var/www/driftpunkt
cd /var/www/driftpunkt
```

2. Kör Debian-hjälpscriptet:

```
DOMAIN=driftpunkt.example.com sudo -E bash deploy/debian/setup.sh
```

Scriptet installerar Apache, PHP-paket, MariaDB, Composer, Apache-vhost, logrotate samt systemd-timers för mailpolling och bilagearkivering. Det skapar också `.env.local` från `deploy/debian/app.env.example` om filen saknas.

3. Redigera produktionsmiljön:

```
sudo nano /var/www/driftpunkt/.env.local
```

Byt minst:

- `APP_SECRET`
- `DEFAULT_URI`
- `DATABASE_URL` med MariaDB och `serverVersion` som innehåller `mariadb`
- `MAILER_DSN` och `MAILER_FROM` när mail ska skickas

4. Skapa MariaDB-databas:

```
sudo mariadb

CREATE DATABASE driftpunkt CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'driftpunkt'@'localhost' IDENTIFIED BY 'byt-databaslösenord';
GRANT ALL PRIVILEGES ON driftpunkt.* TO 'driftpunkt'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

5. Installera eller uppdatera PHP-beroenden:

```
cd /var/www/driftpunkt
sudo -u www-data composer install --no-dev --optimize-autoloader
```

6. Initiera en helt ny databas:

```
sudo -u www-data php bin/console app:install:fresh --env=prod
```

Kommandot skapar schema och säkerställer standardkonton för superadmin och admin. Standardkonton markeras för lösenordsbyte vid första inloggningen. Vill du inte skapa tekniker- och kundkonton:

```
sudo -u www-data php bin/console app:install:fresh --env=prod --skip-test-accounts
```

7. Sätt rättigheter:

```
sudo chown -R www-data:www-data /var/www/driftpunkt/var /var/www/driftpunkt/htdocs/assets
sudo find /var/www/driftpunkt -type d -exec chmod 755 {} \;
sudo find /var/www/driftpunkt -type f -exec chmod 644 {} \;
```

8. Ladda om Apache:

```
sudo apache2ctl configtest
sudo systemctl reload apache2
```

9. Lägg på HTTPS, till exempel med Certbot:

```
sudo apt-get install certbot python3-certbot-apache
sudo certbot --apache -d driftpunkt.example.com
```

10. Aktivera update-worker om adminytans uppdateringsflöde ska användas. Se avsnittet `Automatiska uppdateringar med systemd timer` nedan.

Viktiga produktionsfiler

- Appkatalog: `/var/www/driftpunkt`
- Miljö: `/var/www/driftpunkt/.env.local`
- Apache-vhost: `/etc/apache2/sites-available/driftpunkt.conf`
- Webbrot: `/var/www/driftpunkt/htdocs`
- Bakåtkompatibel äldre webbrot: `/var/www/driftpunkt/public`
- Cache/loggar/uploads: `/var/www/driftpunkt/var`
- Loggrotation: `/etc/logrotate.d/driftpunkt`
- Köade koduppdateringar: `/var/www/driftpunkt/var/code_update_runs`
- Mailpolling timer: `driftpunkt-mail-poll.timer`
- Bilagearkivering timer: `driftpunkt-attachment-archive.timer`
- Update-worker timer: `driftpunkt-update-worker.timer`

Bakgrundsjobb

Systemd-timers installeras av hjälpscriptet. Kontrollera dem med:

```
systemctl list-timers 'driftpunkt-*'
systemctl status driftpunkt-mail-poll.timer
systemctl status driftpunkt-attachment-archive.timer
```

Hjälpscriptet installerar de färdiga timerfilerna som finns i `deploy/debian/`. SLA-kontroll, månadsrapporter och update-worker ska antingen hanteras av appens interna fallback eller sättas upp som egna cron/systemd-jobb efter samma mönster.

Köra jobben manuellt:

```
sudo -u www-data php /var/www/driftpunkt/bin/console app:mail:poll --env=prod
sudo -u www-data php /var/www/driftpunkt/bin/console app:archive-ticket-attachments --env=prod
sudo -u www-data php /var/www/driftpunkt/bin/console app:check-ticket-sla --env=prod
```

Automatiska uppdateringar med systemd timer

Adminytans uppdateringsflöde kan skapa en köad update-run i `var/code_update_runs`. För att den inte ska bli liggande som queued ska Debian-servern ha en separat worker som körs av systemd.

Flödet är:

```
Adminyta köar uppdatering
        ↓
systemd timer startar worker
```

```
        ↓
workern hittar första queued JSON-fil
        ↓
workern kör app:code-update:apply-run
        ↓
status blir completed eller failed
```

1. Skapa worker-script

```
sudo nano /usr/local/bin/driftpunkt-run-pending-updates
```

Innehåll:

```
#!/bin/bash
set -euo pipefail

APP_DIR="/var/www/driftpunkt"
cd "$APP_DIR"

RUN_ID=$(sudo -u www-data php -r '
$dir = "var/code_update_runs";
if (!is_dir($dir)) { exit; }

$files = glob($dir . "/*.json");
rsort($files);

foreach ($files as $file) {
    $data = json_decode(file_get_contents($file), true);
    if (($data["status"] ?? "") === "queued") {
        echo basename($file, ".json");
        exit;
    }
}
')

if [ -n "$RUN_ID" ]; then
    echo "Kör Driftpunkt-uppdatering: $RUN_ID"
    sudo -u www-data php bin/console app:code-update:apply-run "$RUN_ID" --env=prod
else
    echo "Ingen köad Driftpunkt-uppdatering hittades."
fi
```

Gör scriptet körbart:

```
sudo chmod +x /usr/local/bin/driftpunkt-run-pending-updates
```

Testa manuellt:

```
sudo /usr/local/bin/driftpunkt-run-pending-updates
```

2. Skapa systemd-service

```
sudo nano /etc/systemd/system/driftpunkt-update-worker.service
```

Innehåll:

```
[Unit]
Description=Driftpunkt pending update worker
After=network.target mariadb.service apache2.service
Wants=mariadb.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/driftpunkt-run-pending-updates
WorkingDirectory=/var/www/driftpunkt
User=root
Nice=5
```

3. Skapa systemd-timer

```
sudo nano /etc/systemd/system/driftpunkt-update-worker.timer
```

Innehåll:

```
[Unit]
Description=Run Driftpunkt pending update worker every minute
```

```
[Timer]
OnBootSec=1min
OnUnitActiveSec=1min
Unit=driftpunkt-update-worker.service

[Install]
WantedBy=timers.target
```

4. Aktivera och kontrollera

```
sudo systemctl daemon-reload
sudo systemctl enable --now driftpunkt-update-worker.timer
systemctl status driftpunkt-update-worker.timer
```

Visa loggar:

```
journalctl -u driftpunkt-update-worker.service -n 100 --no-pager
```

Kör workern direkt vid felsökning:

```
sudo systemctl start driftpunkt-update-worker.service
journalctl -u driftpunkt-update-worker.service -n 50 --no-pager
```

Om ingen uppdatering väntar ska loggen visa:

```
Ingen köad Driftpunkt-uppdatering hittades.
```

Uppgradering

Rekommenderad ordning när ett paketerat uppgraderingsflöde inte används:

```
cd /var/www/driftpunkt
sudo -u www-data php bin/console app:maintenance on --message="Uppgradering pågår" --env=prod
sudo -u www-data composer install --no-dev --optimize-autoloader
sudo -u www-data php bin/console doctrine:migrations:migrate -n --env=prod
sudo -u www-data php bin/console cache:clear --env=prod
sudo -u www-data php bin/console app:maintenance off --env=prod
```

Ta alltid backup på databas och `var/` innan uppgradering.

Uppgraderingspaket som byggs av Driftpunkt innehåller `vendor/`, så Debian-servern ska kunna uppdateras utan att hämta Composer-paket från internet under själva driftfönstret.

Om adminytans uppdateringsflöde används ska `driftpunkt-update-worker.timer` vara aktiv innan uppdateringen köas. Annars kan körningen bli liggande som queued tills den körs manuellt. Kontrollera också att Apache-vhosten pekar på `/var/www/driftpunkt/htdocs` innan första htdocs-baserade releasepaketet appliceras.

Manuell körning av en köad update-run:

```
cd /var/www/driftpunkt
sudo -u www-data php bin/console app:code-update:apply-run RUN_ID --env=prod
```

Kontrollera status:

```
cat /var/www/driftpunkt/var/code_update_runs/RUN_ID.json
```

Rekommenderad release-struktur för mer kontrollerad produktion

För kundmiljöer där rollback är viktigt bör Driftpunkt på sikt driftsättas med release-mappar och symlink:

```
/var/www/driftpunkt/
├── current -> releases/1.0.9
├── releases/
│   ├── 1.0.6
│   └── 1.0.9
└── shared/
    ├── .env.local
    ├── var/
    └── public/uploads/
```

Ett sådant flöde gör det enklare att byta tillbaka till föregående version om en release behöver rullas tillbaka.

Felsökning

Kontrollera PHP och tillägg:

```
php -v
php -m
composer check-platform-reqs
```

Kontrollera loggar:

```
sudo journalctl -u apache2 -n 100 --no-pager
sudo tail -n 100 /var/www/driftpunkt/var/log/prod.log
sudo tail -n 100 /var/www/driftpunkt/var/log/mail-poll.log
sudo tail -n 100 /var/www/driftpunkt/var/log/ticket-attachment-archive.log
sudo journalctl -u driftpunkt-mail-poll.service -n 100 --no-pager
sudo journalctl -u driftpunkt-update-worker.service -n 100 --no-pager
```

Kontrollera timers:

```
systemctl list-timers 'driftpunkt-*'
systemctl status driftpunkt-update-worker.timer
```

Vanliga fel:

- `Composer detected issues in your platform`: PHP-version eller PHP-tillägg saknas.
- HTTP 500 efter deploy: kontrollera `var/log/prod.log`, `.env.local` och databasanslutningen.
- 404 på alla routes: Apache-vhost pekar inte på `htdocs/` eller `mod_rewrite`/`FallbackResource` saknas.
- Inga mail skickas: `MAILER_DSN` är fortfarande `null://null` eller SMTP-uppgifter saknas.
- Uppdatering ligger kvar som queued: kontrollera `driftpunkt-update-worker.timer`, kör `sudo systemctl start driftpunkt-update-worker.service` och läs `journalctl -u driftpunkt-update-worker.service`.
- `Permission denied` vid uppdatering: kontrollera ägarskap på kodfiler och `var/`, särskilt om adminytans uppdateringsflöde används.

NAS Docker setup

Källa: docs/nas_docker_setup.md

NAS-installation med Docker Compose

Den här guiden är för att sätta upp Driftpunkt på nytt på en NAS efter att containrar, volymer eller delade mappar försvunnit. Den använder Docker Compose och sparar databas samt appens `var/` i synliga mappar under `deploy/nas/data/`.

Krav

- NAS med Docker/Container Manager och Docker Compose
- shell-access via SSH rekommenderas
- Driftpunkt-koden kopierad eller klonad till NAS:en
- MariaDB via den medföljande compose-stacken eller extern MariaDB 10.11+
- `DATABASE_URL` som pekar på MariaDB och anger `serverVersion` med `mariadb`
- appcontainern serverar Driftpunkt från `/var/www/html/htdocs`
- om adminytans uppdateringsflöde ska användas måste appcontainerns `www-data` kunna skriva till applikationskoden i `/var/www/html`, inte bara till `var/`
- om adminytans uppdateringsflöde används bör en NAS-schemalagd uppgift eller cron-worker köra `app:code-update:apply-run` för köade uppdateringar

Efter ombyggd eller ny appcontainer kan kodfilerna behöva ägarrättas innan ett uppdateringspaket appliceras:

```
docker compose -f deploy/nas/compose.yaml --env-file deploy/nas/.env exec -u root app sh -lc
'chown -R www-data:www-data /var/www/html && chmod -R u+rwX /var/www/html'
```

Om du inte använder den medföljande Apache-konfigurationen ska webbservern peka mot `htdocs/`, inte `public/`. Om NAS/webbhotell tvingar hela appen att ligga direkt i en yttre `/htdocs`-katalog behöver root-`.htaccess` vara aktiv så att känsliga appmappar blockeras.

Starta från noll

1. Gå till projektet på NAS:en:

   ```
   cd /sökväg/till/Driftpunkt
   ```

2. Skapa miljofilen:

   ```
   cp deploy/nas/.env.example deploy/nas/.env
   ```

3. Redigera `deploy/nas/.env` och byt minst:

- `APP_SECRET`
- `DEFAULT_URI`
- `MARIADB_PASSWORD`
- `MARIADB_ROOT_PASSWORD`
- samma databaslösenord i `DATABASE_URL`
- `MAILER_DSN` och `MAILER_FROM` när utgående mail ska fungera

4. Bygg och starta:

```
mkdir -p deploy/nas/data/var deploy/nas/data/mariadb
docker compose -f deploy/nas/compose.yaml --env-file deploy/nas/.env up -d --build
```

5. Initiera databasen:

```
docker compose -f deploy/nas/compose.yaml --env-file deploy/nas/.env exec --user www-data
app php bin/console app:install:fresh --env=prod
```

6. Öppna adressen från `DEFAULT_URI`, till exempel:

```
http://nas-ip-eller-domän:8080
```

Standardkonton

`app:install:fresh` säkerställer alltid standardkonton för superadmin och admin. Tekniker- och kundkonton skapas om du inte anger `--skip-test-accounts`:

Roll | E-post | Lösenord

Super admin | `super@test.local` | `AdminPassword123`

Admin | `admin@test.local` | `AdminPassword123`

Tekniker | `tech@test.local` | `TechPassword123`

Kund | `customer@test.local` | `CustomerPassword123`

Alla standardkonton markeras för lösenordsbyte vid första inloggningen.

Skapa hellre en riktig admin efter installationen:

```
docker compose -f deploy/nas/compose.yaml --env-file deploy/nas/.env exec --user www-data app
php bin/console app:create-admin admin@example.com 'BytDettaLösenord123' Förnamn Efternamn
super_admin --env=prod
```

Bakgrundsjobb

Compose-filen startar en `scheduler`-container som kör:

- `app:mail:poll` varje minut
- `app:check-ticket-sla` varje minut
- `app:archive-ticket-attachments` runt 02:15

Köade koduppdateringar från adminytan körs inte av scheduler-containern automatiskt. Använd NAS:ens schemalagda uppgifter eller cron enligt `docs/installation_and_deployment.md` om uppdateringar ska appliceras från adminytan.

Kontrollera loggar:

```
docker compose -f deploy/nas/compose.yaml --env-file deploy/nas/.env logs -f app scheduler
```

Stabilitet

Compose-filen gör följande för en lugnare NAS-drift:

- startar om containrar automatiskt med `restart: unless-stopped`
- väntar på frisk databas innan appen startar fullt
- kör en HTTP-healthcheck mot appen
- rättar ägare på `var/` till `www-data` vid start
- rättar ägare på `htdocs/assets` till `www-data` så brandinguppladdning fungerar
- kör schemalagda Symfony-kommandon som `www-data`

Backup

Ta backup på både databasen och appens `var/`-mapp.

Databasdump:

```
docker compose -f deploy/nas/compose.yaml --env-file deploy/nas/.env exec database mariadb-dump
-u root -p driftpunkt > driftpunkt.sql
```

Samlad backup till `deploy/nas/backups/`:

```
bash deploy/nas/backup.sh
```

Backupscriptet skapar ett arkiv med:

- MariaDB-dump med `--single-transaction`, rutiner, triggers och events
- `deploy/nas/data/var`
- `manifest.txt` med tidpunkt, compose-fil och git-commit när det finns

Det lägger också ett enkelt lås så två backuper inte körs samtidigt, verifierar arkivet med `tar -tzf` och rensar backuper äldre än 14 dagar. Ändra retention vid behov:

```
BACKUP_RETENTION_DAYS=30 bash deploy/nas/backup.sh
```

Restore från ett backup-arkiv:

```
CONFIRM=restore bash deploy/nas/restore.sh deploy/nas/backups/driftpunkt-YYYYMMDD-HHMMSS.tar.gz
```

Restore stoppar app och scheduler, läser in `database.sql`, återställer `data/var` och startar sedan stacken igen. Den gamla `data/var` flyttas till en tidsstamplad `var.before-restore-*`-mapp.

Viktiga mappar:

- `deploy/nas/data/var` för appfiler, bilagor, cache och loggar
- `deploy/nas/data/mariadb` för MariaDB-data

Uppdatering

Ta backup före uppdatering:

```
bash deploy/nas/backup.sh
```

Rekommenderat flöde för paketuppdatering är adminytan:

1. aktivera underhållsläge
2. ladda upp uppgraderingspaketet under Admin -> Uppdateringar
3. applicera versionen och låt förkontroller, MariaDB-migrationer och cache-rensning gå klart
4. verifiera startsida, login, ärenden och mailflöden
5. bekräfta verifieringen så underhållsläget stängs av

Uppgraderingspaket ska innehålla `vendor/`. NAS:en ska därför inte behöva hämta Composer-paket från GitHub under själva uppdateringen.

Manuell uppdatering via shell:

```
git pull
docker compose -f deploy/nas/compose.yaml --env-file deploy/nas/.env up -d --build
docker compose -f deploy/nas/compose.yaml --env-file deploy/nas/.env exec --user www-data app
php bin/console doctrine:migrations:migrate -n --env=prod
docker compose -f deploy/nas/compose.yaml --env-file deploy/nas/.env exec --user www-data app
php bin/console cache:clear --env=prod
```